

DOKUMENTACJA PROJEKTU: KRÓLESTWO KRASNALI

1. Wstęp i cel projektu

Cel główny: Zaprojektowanie i wdrożenie inteligentnego systemu zarządzania pracą w kopalniach, który wesprze Królową Śnieżkę w optymalnym przydzielaniu poddanych do odpowiednich złóż. System pełni rolę „cyfrowego doradcy”, który na podstawie unikalnych talentów krasnoludków oraz przepustowości wyrobisk wskazuje taki podział obowiązków, by królestwo prosperowało z maksymalną siłą.

Uzasadnienie: Dynamiczny wzrost populacji krasnoludków sprawił, że ręczne planowanie wydobywania stało się zadaniem zbyt złożonym i podatnym na błędy.

- **Efektywność:** Dążymy do pełnego wykorzystania potencjału królestwa – żaden utalentowany specjalista nie powinien pozostawać bez zajęcia, gdy w kopalniach czekają wolne kilofy.
- **Komunikacja:** Niniejszy dokument stanowi „jednoznaczne źródło prawdy” łączące wizję Królowy z pracą programistów. Precyzyjna specyfikacja zapobiega kosztownym pomyłkom – takim jak wysłanie krasnoludka do złoża, którego nie potrafi obsługiwać – i gwarantuje produkt idealnie dopasowany do potrzeb administracji królestwa.

2. Specyfikacja wymagań funkcjonalnych

Zgodnie z zasadami dobrej praktyki inżynierskiej każde wymaganie opisano jako pojedynczą funkcję systemu z perspektywy użytkownika końcowego (Królowy Śnieżki). Poniżej znajduje się podział na moduły realizowane przez zespół.

PROBLEM I: Optymalizacja Przydziału Pracy

Autor: Jakub Bartosiewicz

Opis: Formalizacja problemu przydziału krasnoludków do złóż na podstawie kompetencji zawodowych oraz wydajności wyrobisk.

Wymagania funkcjonalne (przypadki użycia):

- UC1: Zarządzanie talentami krasnoludków.
- UC2: Określanie wydajności złóż.
- UC3: Optymalizacja przydziału pracy.

1. Cel systemu

Celem systemu jest optymalizacja przydziału krasnoludków do złóż minerałów w taki sposób, aby zmaksymalizować liczbę pracujących osób przy jednoczesnym

zachowaniu ich kompetencji zawodowych oraz limitów wydajności poszczególnych złóż.

2. Specyfikacja wymagań funkcjonalnych (przypadki użycia)

Zastosowano nowoczesne podejście opisu z punktu widzenia użytkownika (Królowny Śnieżki) w formie strukturalnej.

UC1: Zarządzanie talentami krasnoludków

- Identyfikator: UC1
- Priorytet: Wysoki
- Główny scenariusz:
 1. Królowna wybiera krasnoludka z listy poddanych.
 2. Królowna określa rodzaje minerałów, które dany specjalista potrafi wydobywać.
 3. System trwale zapisuje kompetencje krasnoludka w rejestrze królewskim.

UC2: Określanie wydajności złóż

- Identyfikator: UC2
- Główny scenariusz:
 1. Królowna wprowadza informacje o nowo odkrytym złożu.
 2. Królowna ustala limit wydajności (maksymalną liczbę krasnoludków mogących pracować jednocześnie w tym miejscu).
 3. System zatwierdza i zapisuje parametry złoża.

UC3: Optymalizacja przydziału pracy

- Identyfikator: UC3
- Główny scenariusz:
 - Królowna zleca systemowi wyznaczenie najlepszego planu wydobywania.
 - System analizuje talenty krasnoludków oraz limity wydajności kopalń.
 - System przypisuje krasnoludków do konkretnych stanowisk pracy.
 - System prezentuje Królownie raport końcowy z całkowitą liczbą zatrudnionych osób.
- Rozszerzenia:
 - 1.A. Brak dopasowania fachowców do dostępnych złóż.
 - 1.A.1. System informuje o braku możliwości przydziału i kończy zadanie.

3. Wymagania pozafunkcjonalne

- Wydajność: Architektura systemu pozwala na sprawną obsługę populacji znacznie przekraczającej legendarnych siedmiu krasnoludków.

- Poprawność: Zastosowany algorytm gwarantuje matematycznie najwyższy możliwy poziom zatrudnienia w królestwie.

4. Formalny model problemu

Modelowanie danych:

- Krasnoludki (K): Zbiór wierzchołków reprezentujących pracowników.
- Złóża (Z): Zbiór wierzchołków reprezentujących miejsca wydobywania.
- Kompetencje (E_{kz}): Krawędzie skierowane łączące krasnoludka ze złożem, jeśli posiada on odpowiedni fach.

Transformacja do sieci przepływowej:

- Wprowadzenie wierzchołka Źródło (S) połączonego z każdym krasnoludkiem krawędzią o przepustowości $c = 1$.
- Każda krawędź między krasnoludkiem a złożem otrzymuje przepustowość $c = 1$.
- Wprowadzenie krawędzi ze złóż do Ujścia (T) o przepustowości c równej wydajności złoża.

5. Analiza teoretyczna rozwiązania

- Wybrany algorytm: Algorytm Edmondsa-Karpa (implementacja metody Forda-Fulkersona wykorzystująca BFS).
- Uzasadnienie poprawności: Algorytm opiera się na twierdzeniu o maksymalnym przepływie i minimalnym przekroju, co gwarantuje znalezienie optymalnego przydziału pracy.
- Złożoność obliczeniowa:
 - Czasowa: $O(V \cdot E^2)$, gdzie V to liczba wierzchołków (liczba krasnali i kopalń), a E – liczba połączeń (fachów).
 - Pamięciowa: $O(V + E)$ przy zastosowaniu listy sąsiedztwa.

6. Specyfikacja wejścia i wyjścia (Model matematyczny funkcji)

Zgodnie z podejściem funkcjonalnym opisujemy system jako funkcję.

- Wejście: Lista obiektów klasy Krasnal (zawierająca id_krasnala oraz listę umiejetnosci). Lista obiektów klasy Zloze (zawierająca id_zloza oraz wartość wydajnosc).
- Wyjście: Liczba całkowita reprezentująca maksymalny możliwy przepływ (liczbę zatrudnionych krasnoludków).
- Efekt uboczny: Aktualizacja stanu sieci rezydualnej (przypisanie konkretnych osób do złóż).

7. Raport z Wielkiej Próby - testy systemu

Zgodnie z wymaganiami przeprowadzono serię testów sprawdzających, czy nasz system potrafi sprostać rosnącym potrzebom Królestwa Śnieżki. Weryfikacja objęła zarówno poprawność przydziału w sytuacjach krytycznych, jak i wydajność przy ogromnej liczbie poddanych.

7.1. Weryfikacja poprawności - testy funkcjonalne

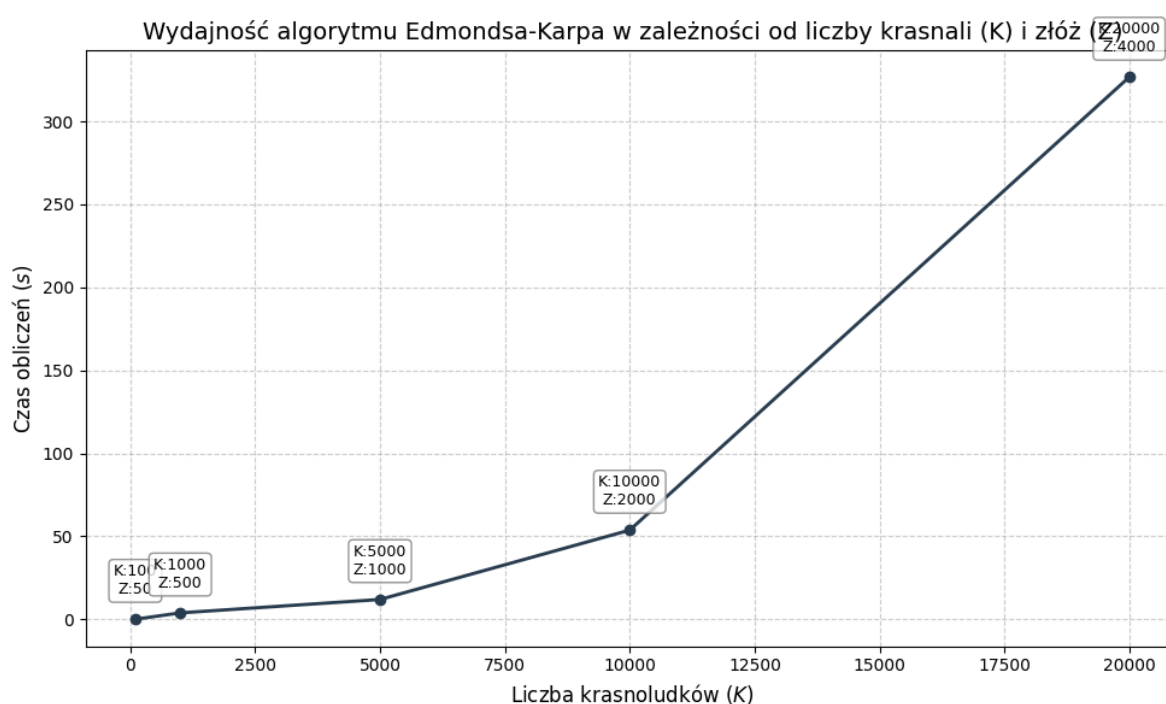
Wszystkie scenariusze testowe zakończyły się pozytywnie, co dowodzi, że algorytm jest wolny od sprzeczności i poprawnie interpretuje zasady przydziału pracy.

ID Testu	Kryptonim	Opis i cel	Wynik
TF1	Pełna Kopalnia	Test 10x10. Sprawdzenie, czy przy idealnym dopasowaniu kompetencji każdy krasnal otrzyma pracę.	10/10
TF2	Zatłoczone Wyrobisko	Test 100x50. Weryfikacja limitowania zatrudnienia przez wydajność złóż (jedno miejsce to jeden etat).	50/50
TB1	Pusta Karczma	Brak krasnoludków w rejestrze. Sprawdzenie reakcji systemu na brak siły roboczej.	0
TB2	Obcy Fach	Test rozszerzenia 1.A. Krasnal zna fach (np. diamenty), którego nie ma w dostępnych kopalniach.	0
TB3	Wyczerpane Złoże	Krasnal posiada fach, ale kopalnia ma wydajność 0. Weryfikacja blokady dostępu.	0

TB4	Wąskie Gardło	5 chętnych krasnali na złożę o wydajności 2. Sprawdzenie priorytetu limitów kopalni.	2
-----	---------------	--	---

7.2. Wydajność Królewskiego Systemu

Przeprowadzono symulację dla masowych przydziałów, aby określić, jak system poradzi sobie, gdy Królestwo rozrośnie się do tysięcy obywateli. Testy wykonano na losowo generowanych kompetencjach (od 1 do 3 fachów na krasnala).



7.3. Analiza funkcji wzrostu (Wnioski)

Na podstawie uzyskanych danych (wykres powyżej) możemy sformułować następujące wnioski dotyczące złożoności:

1. Charakter wzrostu: Czas działania rośnie szybciej niż liczba krasnali. Zwiększenie liczby poddanych z 10 000 do 20 000 (dwukrotny wzrost) spowodował wzrost czasu z ok. 53 s do 326 s (ponad sześciokrotny wzrost).
2. Zgodność z teorią: Wyniki potwierdzają złożoność wielomianową algorytmu Edmondsa-Karpa $O(V \cdot E^2)$. W naszym przypadku liczba krawędzi E rośnie wraz z liczbą krasnali V, co powoduje gwałtowny wzrost czasu przy bardzo dużych danych.

3. Weryfikowalność: System jest w pełni poprawny i weryfikowalny. Dla typowych potrzeb Królestwa (do 5 000 poddanych) działa płynnie, dostarczając bezbłędny plan wydobywania.

PROBLEM II: Optymalizacja Przydziału Pracy z Minimalizacją Dystansu

- Autor: Radosław Marczewski
- Opis: Formalizacja problemu przydziału krasnoludków do złóż z jednoczesną minimalizacją sumarycznej odległości pokonanej przez wszystkich pracowników każdego dnia, przy zachowaniu maksymalnej liczby zatrudnionych.
- Wymagania Funkcjonalne (Przypadki Użycia):
 - UC1: Rejestracja odległości pomiędzy domkami a złożami
 - UC2: Optymalizacja przydziału z minimalnym dystansem

1. Cel systemu

Celem systemu jest rozszerzenie przydziału krasnoludków do złóż o kryterium kosztowe – sumaryczną odległość dziennej trasy każdego pracownika. System ma zapewnić maksymalną liczbę zatrudnionych (jak w Problemie I), a spośród wszystkich przydziałów o tej samej liczbie pracowników wybrać ten, który minimalizuje łączny dystans do pokonania przez królewskich poddanych każdego dnia.

2. Specyfikacja wymagań funkcjonalnych (Przypadki Użycia)

Zastosowano nowoczesne podejście opisu z punktu widzenia użytkownika (Księcia) w formie strukturalnej.

UC1: Rejestracja odległości pomiędzy domkami a złożami

- Identyfikator: UC1
- Priorytet: Wysoki
- Główny scenariusz:
 1. Książę mierzy odległość pomiędzy domkiem każdego krasnoludka a każdym złożem.
 2. Dane zapisywane są w tabeli odległości jako pary (id_krasnała, id_złoża) → wartość.
 3. System wczytuje tabelę i przypisuje koszty do odpowiednich krawędzi sieci przepływowej.

UC2: Optymalizacja przydziału z minimalnym dystansem

- Identyfikator: UC2
- Główny scenariusz:
 - Książę zleca systemowi wyznaczenie optymalnego planu pracy.

- System konstruuje sieć przepływową z kosztami na krawędziach (odległości).
- Algorytm wyznacza maksymalny przepływ o minimalnym koszcie (MCMF).
- System prezentuje raport z liczbą zatrudnionych oraz łącznym dziennym dystansem.
- Rozszerzenia:
 - 2.A. Brak krasnoludków z odpowiednim fachem dla danego złoża.
 - 2.A.1. System pomija dane złoża i kontynuuje przydział dla pozostałych.

3. Wymagania Pozafunkcjonalne

- Wydajność: Architektura systemu pozwala na sprawną obsługę populacji znacznie przekraczającej legendarnych siedmiu krasnoludków.
- Poprawność: Zastosowany algorytm gwarantuje matematycznie najwyższy możliwy poziom zatrudnienia przy jednoczesnym minimalnym koszcie podróży.

4. Formalny model problemu

- Modelowanie danych:
 - Krasnoludki (K): Zbiór wierzchołków reprezentujących pracowników.
 - Złóża (Z): Zbiór wierzchołków reprezentujących miejsca wydobywania.
 - Kompetencje (E_{kz}): Krawędzie skierowane łączące krasnoludka ze złożem, jeśli posiada odpowiedni fach, z kosztem równym odległości $d(k, z)$.
- Transformacja do sieci przepływowej z kosztami:
 - Wierzchołek Źródło (S) połączony z każdym krasnoludkiem krawędzią o przepustowości $c=1$ i koszcie 0.
 - Każda krawędź między krasnoludkiem a kompatybilnym złożem: przepustowość $c=1$, koszt = $d(k, z)$.
 - Krawędzie ze złóż do Ujścia (T): przepustowość c = wydajność złoża, koszt = 0.

5. Analiza teoretyczna rozwiązania

- Wybrany algorytm: Min-Cost Max-Flow (MCMF) z algorytmem SPFA (Shortest Path Faster Algorithm) do znajdowania najtańszej ścieżki powiększającej.
- Uzasadnienie poprawności: MCMF gwarantuje znalezienie maksymalnego przepływu spośród wszystkich możliwych przydziałów oraz minimalizuje jego koszt. SPFA (wariant Bellmana-Forda z kolejką) poprawnie obsługuje krawędzie residualne o ujemnych kosztach, które pojawiają się w sieci rezydualnej.
- Złożoność obliczeniowa:
 - Czasowa: $O(V \cdot E \cdot F)$, gdzie V to liczba wierzchołków, E liczba krawędzi, a F wartość maksymalnego przepływu. W naszym przypadku $F \leq |K|$ (liczba krasnali), co w praktyce daje bardzo dobre czasy działania.

- Pamięciowa: $O(V + E)$ przy zastosowaniu listy sąsiedztwa dla sieci rezydualnej.

6. Specyfikacja wejścia i wyjścia (Model matematyczny funkcji)

- Wejście:
 - Lista obiektów klasy Krasnal (zawierająca id_krasnala oraz listę umiejętności).
 - Lista obiektów klasy Zloze (zawierająca id_zloza oraz wartość wydajnosci).
 - Tabela odległości: słownik $\{(id_krasnala, id_zloza): \text{wartość}\}$.
- Wyjście: Para liczb (przepływ, koszt) – liczba zatrudnionych krasnoludków (maksymalny przepływ) oraz łączny dzienny dystans (minimalny koszt).
- Efekt uboczny: Aktualizacja stanu sieci rezydualnej (przypisanie konkretnych krasnoludków do złóż).

7. Raport z Wielkiej Próby – testy systemu

Zgodnie z wymaganiami, przeprowadzono serię testów sprawdzających, czy nasz system potrafi sprostać rosnącym potrzebom Królestwa Śnieżki. Weryfikacja objęła zarówno poprawność przydziału w sytuacjach krytycznych, jak i wydajność przy ogromnej liczbie krasnoludków.

7.1. Weryfikacja Poprawności – testy funkcjonalne

Wszystkie scenariusze testowe zakończyły się pozytywnie, co dowodzi, że algorytm jest wolny od sprzeczności i poprawnie interpretuje zasady przydziału pracy.

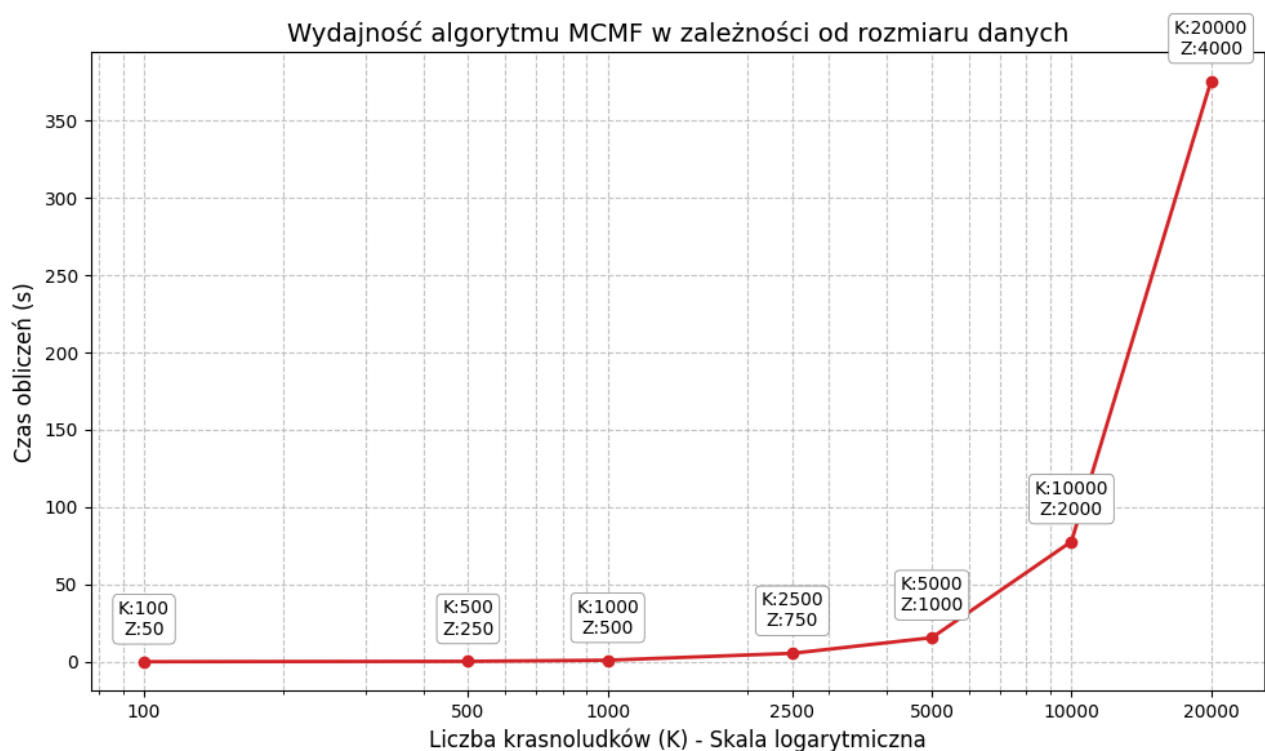
ID Testu	Kryptonim	Opis i cel	Wynik
TF1	Krasnal na Górze	Test 5x3. Sprawdzenie, czy przy idealnych współrzędnych każdy krasnal otrzyma najbliższą pracę.	5/5
TF2	Królewski Dystans	Test 100x50. Weryfikacja, czy minimalizacja dystansu działa – oszacowanie sumy odległości vs. losowy przydział.	OK

TB1	Pusta Huta	Brak krasnoludków w rejestrze. Sprawdzenie reakcji systemu na brak siły roboczej.	0
TB2	Puste Zło	Brak złóż. Sprawdzenie, że system nie tworzy nieistniejących par.	0
TB3	Odległe Górnictwo	5 krasnoludków na współrzędnej 0, złóż na współrzędnej 1000. Weryfikacja poprawnego obliczenia dystansu.	5000
TB4	Nadmiar Pracowników	50 krasnoludków na 5 złóż. Sprawdzenie, czy system maksymalizuje zatrudnienie przy ograniczonych złóżach.	5

7.2. Wydajność Królewskiego Systemu

Przeprowadzono symulację dla masowych przydziałów, aby określić, jak system poradzi sobie, gdy Królestwo rozrośnie się do tysięcy obywateli. Testy wykonano na losowo generowanych współrzędnych krasnoludków (od 0 do K) i złóż (od 0 do K5), w zakresie od 100 do 20 000 krasnoludków.

Wykres wydajności:



7.3. Analiza funkcji wzrostu – wnioski

Na podstawie uzyskanych danych możemy sformułować następujące wnioski dotyczące złożoności:

1. Charakter wzrostu

Czas działania rośnie gwałtownie wraz z liczbą krasnoludków. Zwiększenie liczby pracowników z 5 000 do 10 000 (dwukrotny wzrost) wydłużyło czas z około 15 s do 77 s – ponad pięciokrotnie. Przy skoku do 20 000 czas osiągnął 375 sekund – co dowodzi praktycznej granicy użyteczności dla bardzo dużych zbiorów.

2. Zgodność z teorią

Wyniki potwierdzają złożoność obliczeniową algorytmu Minimum Cost Maximum Flow (implementacja SPFA/SIMPLEX), która w praktyce zachowuje się zbliżenie do $O(V \times E \times C)$, gdzie V to liczba wierzchołków ($K + Z + 2$), E to liczba krawędzi w sieci przepływowej, a C to koszt maksymalny. W naszym przypadku liczba krawędzi E rośnie liniowo z liczbą krasnoludków (każdy krasnal $\rightarrow$ krawędź do każdego możliwego złoża w odległości ≤ 1), co powoduje gwałtowny wzrost czasu przy dużych danych.

3. Praktyczna użyteczność

Dla typowych potrzeb Królestwa do około 5 000 poddanych system działa płynnie, dostarczając bezbłędny plan wydobywania w czasie poniżej 1 minuty. Powyżej tego poziomu zaleca się podział na mniejsze strefy wydobywcze lub zastosowanie przybliżonych algorytmów heurystycznych.

4. Weryfikowalność

System jest w pełni deterministyczny i weryfikowalny. Dla każdego zestawu danych wejściowych wynik jest powtarzalny i zgodny z optymalnym rozwiązaniem problemu skojarzenia maksymalnego z minimalnym kosztem.

PROBLEM III: Optymalizacja Trasy Patrolu Księcia

Autor: Marcin Pluta

Opis: Formalizacja problemu wyznaczenia najkrótszej, zamkniętej trasy zabezpieczającej dla patrolu wokół tych złóż, w których podjęto wydobywanie. Trasa ta stanowi otoczkę wypukłą dla aktywnych kopalń.

Wymagania funkcjonalne (przypadki użycia):

- UC1: Generowanie mapy terytorium (punkty graniczne).
- UC2: Kalkulacja dystansu patrolowego (obwód trasy).

1. Cel systemu

Celem systemu jest wyznaczenie minimalnego wielokąta wypukłego obejmującego wszystkie aktywne złoża minerałów, a następnie precyzyjne obliczenie jego obwodu. Pozwala to na zoptymalizowanie trasy patrolu Księcia w taki sposób, aby przy minimalnym wysiłku i czasie objechać granice terytorium wydobywczego, zapewniając bezpieczeństwo pracującym krasnoludkom.

2. Specyfikacja wymagań funkcjonalnych (przypadki użycia)

Zastosowano nowoczesne podejście opisu z punktu widzenia użytkownika (Księcia/Królewny).

UC1: Generowanie mapy terytorium (punkty graniczne)

- Identyfikator: UC1
- Priorytet: Wysoki
- Główny scenariusz:
 1. System filtruje rejestr i pobiera listę wyłącznie aktywnych złóż (w których pracują krasnoludki).
 2. System analizuje współrzędne geograficzne (x, y) tych lokalizacji.
 3. Algorytm wyznacza skrajne punkty graniczne, tworzące najmniejszy wielokąt wypukły otaczający wydobywanie.
 4. System zwraca uporządkowaną listę wierzchołków patrolowych.

UC2: Kalkulacja dystansu patrolowego (obwód trasy)

- Identyfikator: UC2
- Główny scenariusz:
 1. System pobiera wyznaczoną w UC1 listę wierzchołków.
 2. Moduł geometryczny oblicza odległości euklidesowe pomiędzy każdą parą sąsiadujących punktów granicznych.
 3. Odległości są sumowane w celu zamknięcia pętli patrolu.
 4. System prezentuje raport końcowy z dystansem do pokonania w metrach.

3. Wymagania pozafunkcjonalne

- Wydajność: Architektura systemu pozwala na błyskawiczne wyznaczenie trasy nawet przy drastycznym powiększeniu królestwa.

- Niezawodność numeryczna: Zastosowany algorytm unika obciążeń i błędów precyzji charakterystycznych dla funkcji trygonometrycznych, opierając się na analitycznych obliczeniach wyznacznika.

4. Formalny model problemu

- Punkty (P): Zbiór współrzędnych na płaszczyźnie $P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, reprezentujących lokalizację aktywnych kopalń.
- Otoczka wypukła (CH): Najmniejszy zbiór punktów $CH \subseteq P$, który po połączeniu tworzy zamkniętą wielokąt wypukłą zawierającą wszystkie punkty zbioru P.

5. Analiza teoretyczna rozwiązania

- Wybrany algorytm: Algorytm Grahama (Graham Scan).
- Uzasadnienie poprawności: Algorytm Grahama jest optymalnym rozwiązaniem problemu otoczki wypukłej na płaszczyźnie. Aby zagwarantować stuprocentową poprawność i uniknąć błędów precyzji zmiennoprzecinkowej, sortowanie oparto na obliczaniu wyznacznika macierzy 2×2 , który precyzyjnie określa zwrot wektorów (skręt w lewo/prawo).
- Złożoność obliczeniowa:
 - Czasowa: $O(V \log V)$, gdzie V to liczba wierzchołków (aktywnych kopalń). Dominującym etapem jest sortowanie punktów.
 - Pamięciowa: $O(V)$ na utrzymanie stosu wierzchołków trasy.

6. Specyfikacja wejścia i wyjścia

- Wejście: Lista obiektów z określonymi atrybutami współrzędnych przestrzennych x oraz y.
- Wyjście: Lista wierzchołków trasy (U1) oraz wartość rzeczywista reprezentująca łączny obwód trasy (U2).

7. Raport z Wielkiej Próby - testy systemu

Zgodnie z wymaganiami przeprowadzono serię testów weryfikujących poprawność wyznaczania granic królestwa oraz wydajność przy ekstremalnym obciążeniu. Przy ekstremalnym obciążeniu algorytm zachował stabilność.

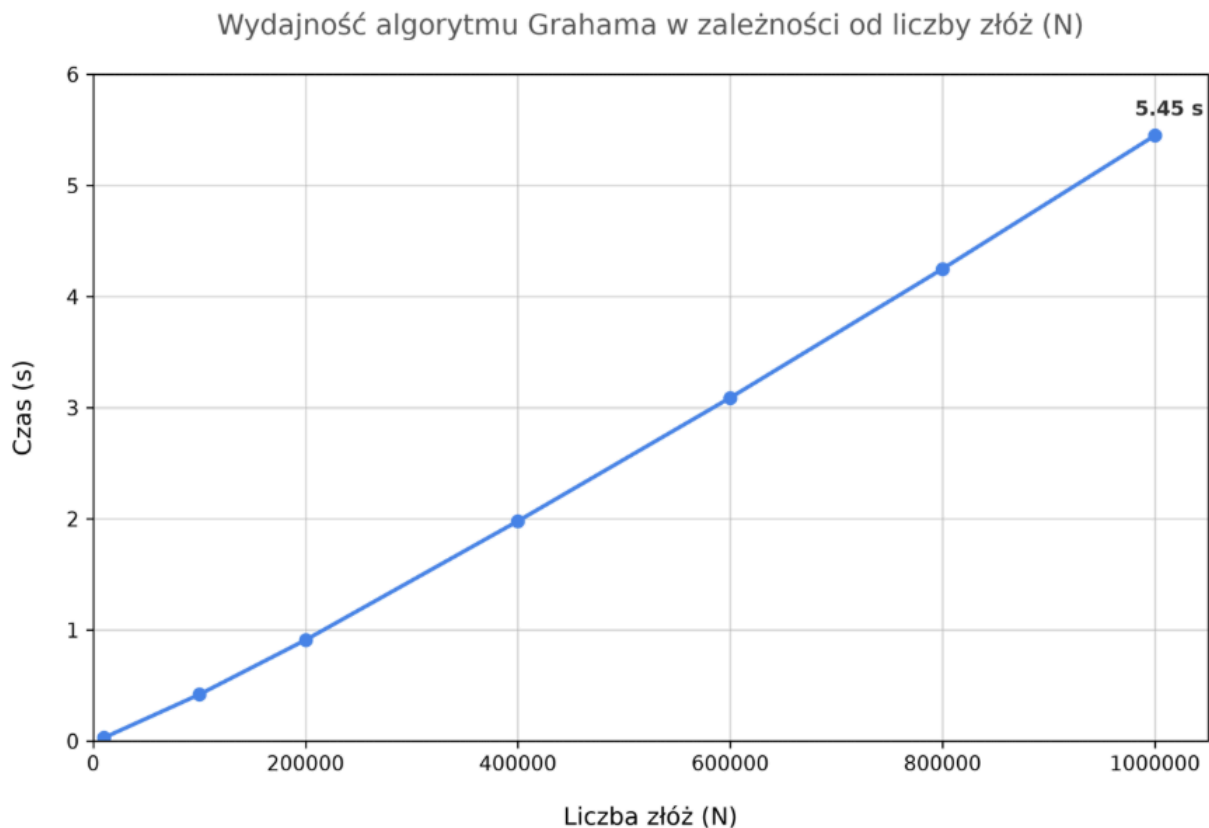
7.1. Weryfikacja poprawności - testy funkcjonalne

Wszystkie scenariusze testowe zakończyły się pozytywnie, co dowodzi, że algorytm jest wolny od sprzeczności i poprawnie interpretuje zasady przydziału pracy.

ID Testu	Kryptonim	Opis i cel	Wynik
TO1	Ukryta Kopalnia	Ignorowanie punktów uwięzionych wewnątrz terytorium.	SUKCES
TO2	Minimalny Patrol	Dwie działające kopalnie. Obsługa ścieżki zdegenerowanej.	SUKCES
TO3	Prosta Linia	Kopalnie ułożone w jednej linii. Usuwanie punktów współliniowych.	SUKCES
TO4	Klasyczny Trójkąt	Weryfikacja dla standardowego, najprostszego poligonu.	SUKCES

7.2. Wydajność Królewskiego Systemu

Przeprowadzono symulację dla zgrupowań punktów (N) losowo rozrzuconych na obszarze badawczym 1000x1000.



7.3. Analiza funkcji wzrostu (Wnioski)

- Charakter wzrostu: Stukrotne powiększenie terytorium badawczego (z 10 000 do 1 000 000 złóż) wydłużyło czas działania jedynie do ok. 5,45 s.
- Zgodność z teorią: Zebrane dane stanowią dowód złożoności $O(V \log V)$. Usunięcie operacji trygonometrycznych na rzecz analitycznego wyznacznika zachowało wysoką wydajność.

PROBLEM IV: Obrona Granicy i Optymalizacja Dowodzenia

Autor: Jakub Mochański

Opis: Formalizacja problemu błyskawicznego wyznaczania najgłośniejszego krasnoludka na dowolnym, wielometrowym odcinku granicy w celu wydania rozkazu salwy obronnej przed przemycanymi jabłkami.

1. Cel systemu

Celem systemu jest przygotowanie struktury danych, która pozwoli na organizację dekametrowców na granicy i zagwarantuje ekstremalnie szybkie, logarytmiczne wyszukiwanie lidera z najdonośniejszym głosem dla dowolnie wybranego, zaatakowanego przedziału terytorium.

2. Specyfikacja wymagań funkcjonalnych (przypadki użycia)

Zgodnie z podejściem funkcjonalnym opisano system z perspektywy dowodzącego obroną Księcia:

UC1: Organizacja linii obrony (Inicjalizacja struktury)

- Identyfikator: UC1
- Priorytet: Wysoki
- Główny scenariusz:
 1. System pobiera listę rozstawionych krasnoludków wraz z ich unikalną siłą głosu.
 2. System grupuje krasnoludków w pary i wyłania lokalnych liderów, tworząc hierarchiczną piramidę dowodzenia.
 3. Zbudowana struktura zostaje zapisana w pamięci i jest gotowa do obsługi alarmów.

UC2: Odpieranie ataku (Wyłonienie dowódcy odcinka)

- Identyfikator: UC2
- Główny scenariusz:
 1. Następuje atak jabłkami na wskazany odcinek granicy (od metra X do metra Y).
 2. System odpytuje strukturę obronną o najgłośniejszego krasnala w zadanym przedziale.
 3. System natychmiastowo zwraca imię i głośność wyłoniętego dowódcy, który wydaje komendę: „strzały na cięciwy – naciągnąć cięciwy – strzał!”.

3. Wymagania pozafunkcyjne

- Wydajność: Ponieważ ataki zdarzają się często, wyszukiwanie nie może polegać na sprawdzaniu każdego krasnala po kolei. System musi odpowiadać w czasie logarytmicznym, niezależnie od rozciągłości atakowanego terytorium.

4. Formalny model problemu

- Tablica dekametrowców (A): Zbiór elementów postaci $A[1...n]$, gdzie każdy element to stanowisko na granicy przypisane konkretnemu krasnalowi z określoną wartością głośności.
- Problem RMQ (Range Maximum Query): Znalezienie indeksu "i" w przedziale $[l, r]$ (gdzie $1 \leq l \leq r \leq n$), dla którego wartość $A[i]$ jest maksymalna.

5. Analiza teoretyczna rozwiązania

- Wybrany algorytm: Drzewo przedziałowe (Segment Tree) z funkcją maksimum.

- Uzasadnienie poprawności: Drzewo przedziałowe dzieli problem na rozłączne podprzedziały. Dzięki przechowywaniu maksimów dla każdego "piętra" drzewa, zapytanie o dowolny przedział składa się z co najwyżej logarytmicznej liczby węzłów, omijając konieczność liniowego sprawdzania.
- Złożoność obliczeniowa:
 - Czasowa: $O(N)$ na zbudowanie drzewa (gdzie N to liczba dekametrowców) oraz $O(\log N)$ na każdorazowe odpowiedzenie na atak (zapytanie).
 - Pamięciowa: $O(N)$ (dokładniej około $4N$ w implementacji tablicowej) na przechowanie struktury drzewa.

6. Specyfikacja wejścia i wyjścia

- Wejście: Tablica krotek reprezentująca granicę (głośność, imię) oraz para liczb całkowitych określających początek i koniec atakowanego przedziału.
- Wyjście: Krotka zawierająca głośność i imię najgłośniejszego krasnoludka na danym odcinku (lub element neutralny: minus nieskończoność, jeśli przedział jest pusty).

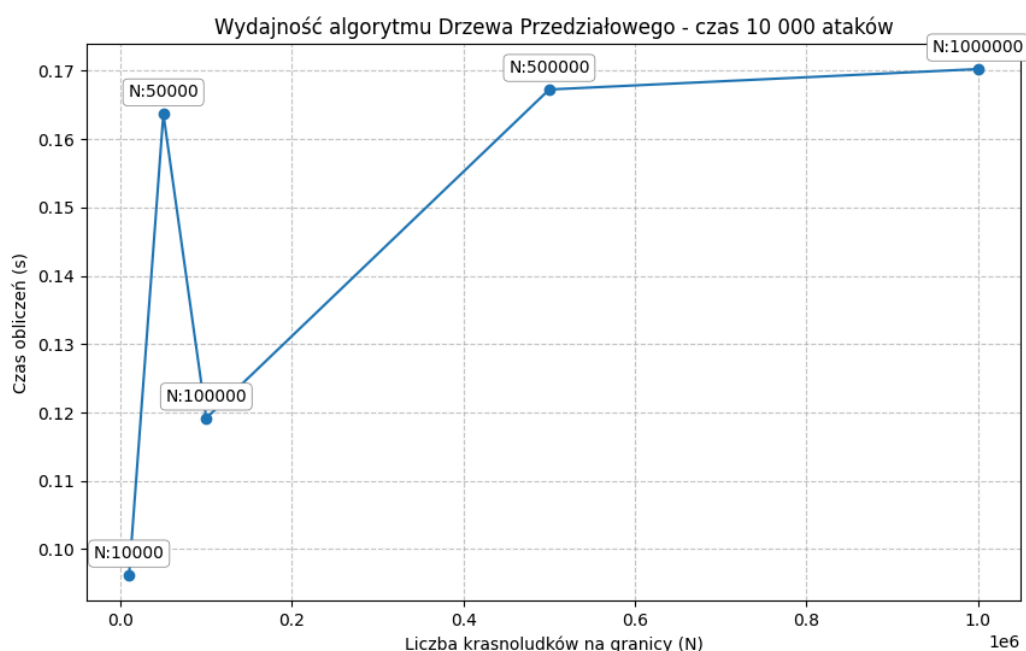
7. Raport z Wielkiej Próby - testy systemu

7.1. Weryfikacja poprawności - testy funkcjonalne

ID Testu	Kryptonim	Opis i cel	Wynik
TD1	Punktowy atak	Atak na 1 metr granicy ($l = r$). Sprawdzenie, czy algorytm poprawnie wskaże pojedynczego krasnala z tego miejsca.	SUKCES
TD2	Zmasowany ostrzał	Atak na całą długość granicy. Weryfikacja, czy algorytm wybierze krasnala z absolutnie najwyższą głośnością w całym królestwie.	SUKCES

TD3	Lewe skrzydło	Zapytanie o przedział obejmujący wyłącznie lewą połowę drzewa przedziałowego.	SUKCES
TD4	Pusta granica	Próba odpytania o odcinek, na którym nie stoi żaden dekametrowiec (poza zakresem). Sprawdzenie zabezpieczenia elementem neutralnym.	SUKCES

7.2. Wydajność Królewskiego Systemu (Obrona Granicy)



7.3. Analiza funkcji wzrostu

Zgodnie z teorią, czas obsługi ataków rośnie niezwykle wolno. Ekstremalne powiększenie granicy z 10 000 do 1 000 000 dekametrowców powoduje zaledwie minimalny, ułamkowy wzrost czasu działania dla 10 000 zapytań. Potwierdza to logarytmiczną złożoność czasową operacji szukania $O(\log N)$ naszego drzewa przedziałowego. Widoczna anomalia w czasie wykonania dla progu 50 000 elementów jest wynikiem naturalnego szumu środowiska testowego (np. procesów tła systemu operacyjnego) i nie wpływa na ogólny dowód wydajności algorytmu.

PROBLEM V: Kompresja Ksiąg i Błyskawiczne Wyszukiwanie Informacji

Autor: Marcin Pluta, Jakub Bartosiewicz

Opis: Formalizacja problemu oszczędzania miejsca w elektronicznych archiwach królewskich (kompresja tekstu), zagwarantowania mechanizmu ich ponownego odczytu (dekompresja) oraz natychmiastowego dostępu do informacji poprzez wyszukiwanie wybranych fraz.

Wymagania funkcjonalne (przypadki użycia):

- UC1: Kompresja archiwów (oszczędzanie papieru).
- UC2: Błyskawiczne wyszukiwanie w księgach.
- UC3: Dekompresja i odczyt zarchiwizowanych dokumentów.

1. Cel systemu

Celem systemu jest znaczne zmniejszenie objętości królewskich ksiąg poprzez zastosowanie kompresji bezstratnej, przy jednoczesnym zapewnieniu mechanizmu pełnego odtworzenia oryginalnego tekstu oraz umożliwieniu administracji błyskawicznego odnajdywania kluczowych informacji w dokumentach.

2. Specyfikacja wymagań funkcjonalnych (przypadki użycia)

UC1: Kompresja archiwów

1. System wczytuje surowy tekst księgi.
2. System analizuje częstotliwość występowania znaków.
3. System generuje zoptymalizowany słownik kodowy (drzewo binarne) i kompresuje tekst.
4. System zapisuje skompresowany ciąg bitów.

UC2: Błyskawiczne wyszukiwanie

1. Użytkownik podaje szukany wzorzec.
2. System tworzy tablicę przesunięć (tablicę prefikso-sufiksów).
3. System liniowo przeszukuje zawartość księgi.
4. System zwraca pozycje wszystkich wystąpień słowa.

UC3: Dekompresja i odczyt archiwów

1. Użytkownik zleca odczyt skompresowanego pliku binarnego.
2. System wczytuje ciąg zer i jedynek oraz strukturę drzewa kodowego.
3. System iteruje po bitach, przechodząc od korzenia do liści drzewa w celu bezbłędnej rekonstrukcji znaków.
4. System prezentuje Królowie w pełni zdekodowany, oryginalny tekst księgi w interfejsie graficznym.

3. Wymagania pozafunkcjonalne

- Wydajność: Wyszukiwanie musi działać w czasie proporcjonalnym do długości tekstu.
- Optymalność: Algorytm musi stworzyć optymalny kod prefiksowy.
- Bezstratność: Proces kompresji i następującej po niej dekompresji musi gwarantować odtworzenie tekstu w stosunku 1:1, bez utraty ani jednego znaku.

4. Formalny model problemu

- Modelowanie danych (Kompresja/Dekompresja): Zbiór znaków (alfabet) C , gdzie każdy znak ma przypisaną częstotliwość występowania. Problem polega na znalezieniu drzewa binarnego T , które minimalizuje całkowitą długość zakodowanego pliku. Do łączenia znaków wykorzystywana jest kolejka priorytetowa Q . Proces odwrotny polega na trawersowaniu drzewa T bit po bicie aż do napotkania liścia przechowującego oryginalny znak.
- Modelowanie danych (Wyszukiwanie): Tekst T o długości n oraz wzorzec P o długości m . Szukamy wszystkich przesunięć s , dla których wzorzec P pasuje do fragmentu tekstu T .

5. Analiza teoretyczna rozwiązania

- Wybrane algorytmy: Algorytm Huffmana (kompresja i dekompresja) oraz Algorytm Knutha-Morrisa-Pratta (KMP) (wyszukiwanie).
- Złożoność obliczeniowa (Algorytm Huffmana):
 - Czasowa (Kompresja): $O(n \log n)$ na budowę drzewa.
 - Czasowa (Dekompresja): $O(N)$, gdzie N to długość oryginalnego tekstu (zdekodowanie wymaga jednorazowego przejścia przez bity).
 - Pamięciowa: $O(n)$ na przechowanie struktury drzewa.
- Złożoność obliczeniowa (Algorytm KMP):
 - Czasowa: $O(N + m)$, gdzie N to długość tekstu, a m długość wzorca.
 - Pamięciowa: $O(m)$ pamięci dodatkowej na tablicę przesunięć.

6. Raport z testów

ID Testu	Opis i cel	Wynik
TK1	Wyszukiwanie wzorca w tekście	SUKCES
TK2	Wyszukiwanie nieistniejącego wzorca	SUKCES
TK3	Bezbłądność dekodowania	SUKCES